

Informe Técnico de Evaluación de Seguridad

Microservicio Backend

Contexto

Este informe presenta un análisis de seguridad del microservicio desarrollado con FastAPI y SQLite. La evaluación se realiza en función del enunciado **Ejercicio Práctico 1: Análisis de Seguridad y Mitigación en un Módulo de Administración**, que contempla cuatro escenarios críticos de vulnerabilidad.

Evaluación por Casos

Caso 1 – Entrada sin validación y salida sin escape (XSS)

Ruta afectada:

```
@app.get("/search")
```

```
def search(q: str):
```

```
    return PlainTextResponse(f"Resultados para: {q}")
```

Vulnerabilidad identificada:

El parámetro `q` se imprime directamente en la respuesta sin ninguna validación o escape, lo que permite la ejecución de scripts maliciosos (ataques de tipo **Cross-Site Scripting - XSS**).

Riesgo asociado:

Alta probabilidad de ejecución de código en el navegador del usuario. Puede ser explotado para robar cookies, realizar acciones en nombre del usuario o desplegar contenido fraudulento.

Medidas de mitigación:

- Validar y sanitizar entradas del usuario.
- Escapar adecuadamente la salida en HTML (`html.escape`, `bleach`, etc.).
- Usar cabeceras como `Content-Type: text/plain` de forma estratégica.

Caso 2 – Exportación sin autenticación ni control de acceso

Ruta afectada:

```
@app.get("/admin/users/export")
```

```
def export_users(db: Session = Depends(get_db)):
```

```
...
```

Vulnerabilidad identificada:

Este endpoint expone información sensible de usuarios (ID, nombre y correo electrónico) sin requerir autenticación ni validación de rol.

Riesgo asociado:

Cualquier actor externo puede acceder y descargar información confidencial sin ningún tipo de restricción, violando principios básicos de confidencialidad y autorización.

Medidas de mitigación:

- Implementar autenticación por token (JWT, OAuth2, sesiones).
- Aplicar control de acceso basado en roles (RBAC).
- Registrar y auditar accesos a recursos críticos.

Caso 3 – Uso de bibliotecas potencialmente vulnerables (frontend)

Observación:

En el backend no se identifican dependencias JavaScript inseguras. Sin embargo, este caso puede aplicarse si el frontend utiliza librerías obsoletas o vulnerables como `jQuery`, `Bootstrap`, `axios`, entre otras.

Riesgo asociado:

Librerías con CVEs conocidos pueden facilitar ataques client-side, escaladas de privilegio o ejecución remota de código.

Medidas de mitigación:

- Auditar periódicamente con herramientas como `npm audit`, `snyk`, `OWASP Dependency Check`.
- Establecer un ciclo de actualización y pruebas de seguridad continuas.

Caso 4 – Eliminación de usuarios sin protección contra CSRF

Ruta afectada:

```
@app.post("/delete-user")
```

```
async def delete_user(request: Request, db: Session = Depends(get_db)):
```

Vulnerabilidad identificada:

No se implementa ninguna protección contra **Cross-Site Request Forgery (CSRF)**, lo que permite que sitios externos realicen peticiones autenticadas en nombre del usuario.

Riesgo asociado:

Acciones críticas como la eliminación de usuarios pueden ejecutarse desde sitios maliciosos si el usuario tiene una sesión activa, comprometiendo la integridad del sistema.

Medidas de mitigación:

- Implementar tokens CSRF únicos por sesión o formulario.
- Validar cabeceras **Origin** y **Referer**.
- Cambiar el método HTTP a **DELETE** y reforzar validaciones.

Resumen de Evaluación

Aspecto Evaluado	Resultado
Validación de entradas	✗ Inexistente en /search
Control de acceso a datos	✗ Inexistente en /admin/users/export
Protección contra CSRF	✗ Inexistente en /delete-user
Uso de dependencias seguras	⚠ No evaluado (requiere revisar el frontend)

Preparado para prácticas
ofensivas

✓ Completamente adecuado

Conclusión

El microservicio backend analizado presenta vulnerabilidades críticas que permiten simular escenarios reales de ataques web como **XSS**, **CSRF**, y exposición de datos sin autenticación. Estas debilidades lo convierten en un entorno ideal para ejercicios educativos, auditorías éticas y prácticas de ofensiva controlada.

